

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ

ВЫСШЕГО ОБРАЗОВАНИЯ

«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Кафедра Вычислительной техники

ОТЧЁТ

по лабораторной работе № 5

«Изучение фильтров и функции Canny для выявления границ»

по дисциплине: «Математические методы распознавания образов»

Выполнили:

Студенты гр. АПИМ-24, АВТФ:

Чернощеков Владимир Сергеевич

Преподаватель:

Ильиных Сергей Петрович

Новосибирск, 2025

Задание:

- Изучить программу и ее возможности.
 - Изучить особенности матричных фильтров для выявления границ и размытия/повышения контрастности изображения.
 - Довести до рабочего состояния программу, приведенную ниже.
- Провести 2-3 эксперимента с разными значениями параметров (порогов) функции Canny()

Ход работы:

Интерфейс программы:

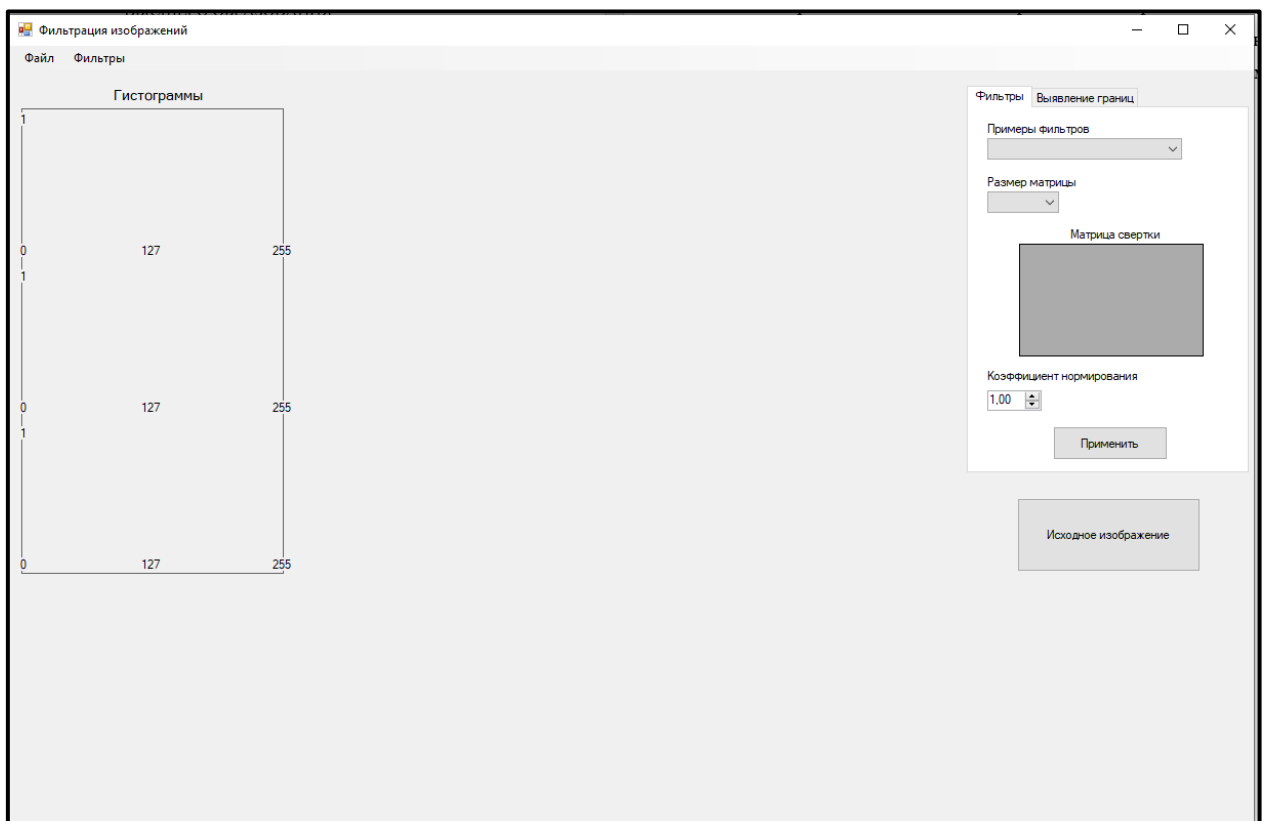


Рисунок 1 – Рабочее пространство программы

Фильтр размытия:

До применения фильтра:

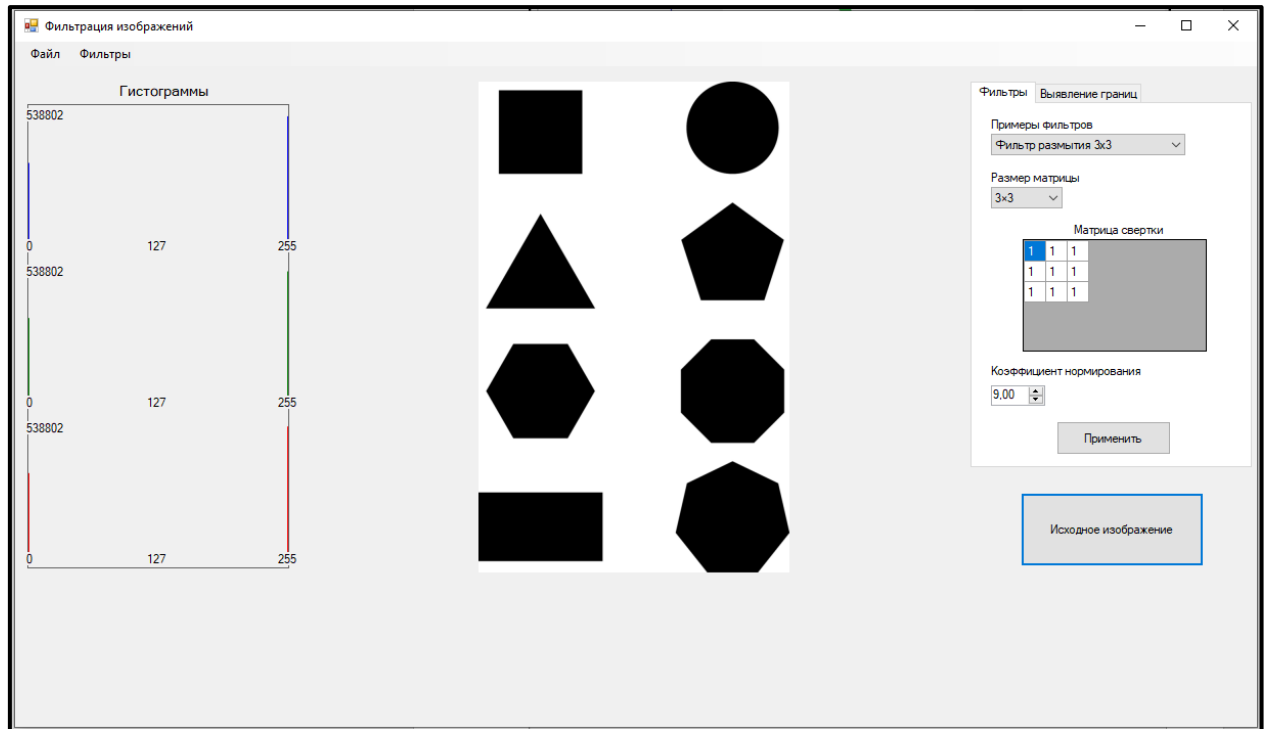


Рисунок 2 – Оригинальное изображение

После применения фильтра:

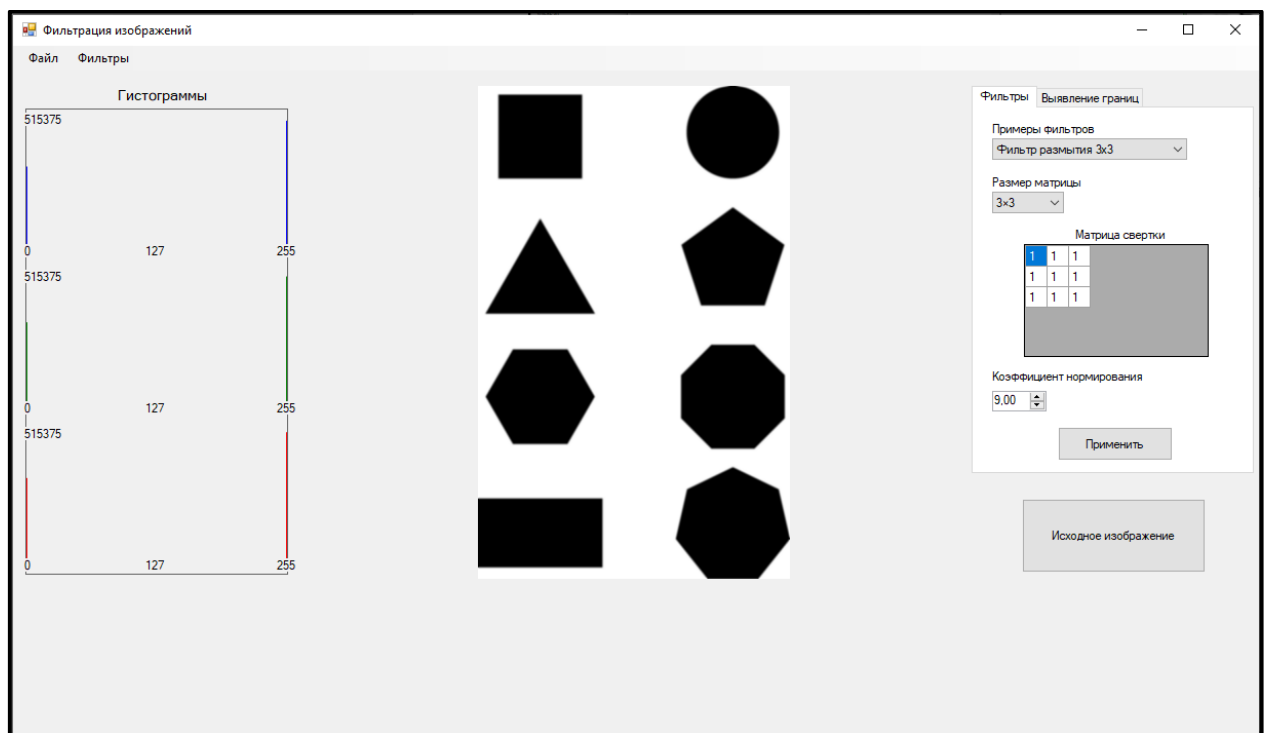


Рисунок 3 – Применяем фильтр размытия

Фильтр резкости:

До применения фильтра:

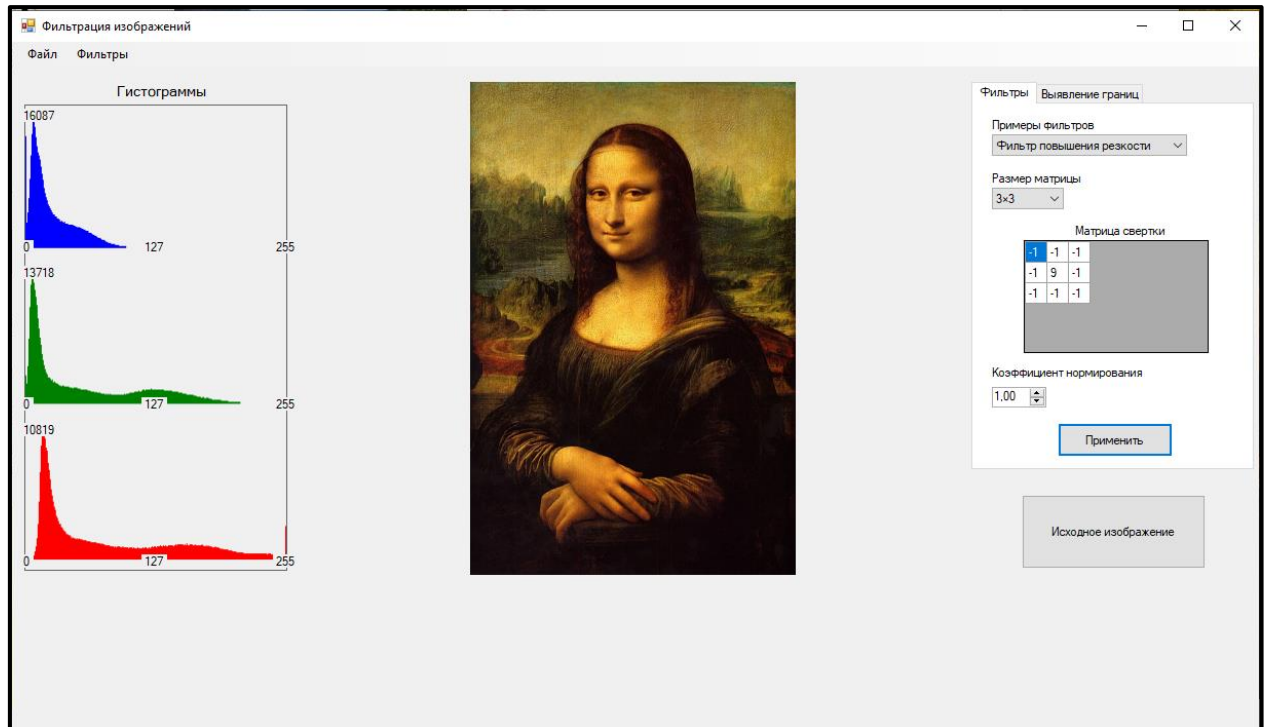


Рисунок 4 – Оригинальное изображение

После применения фильтра:

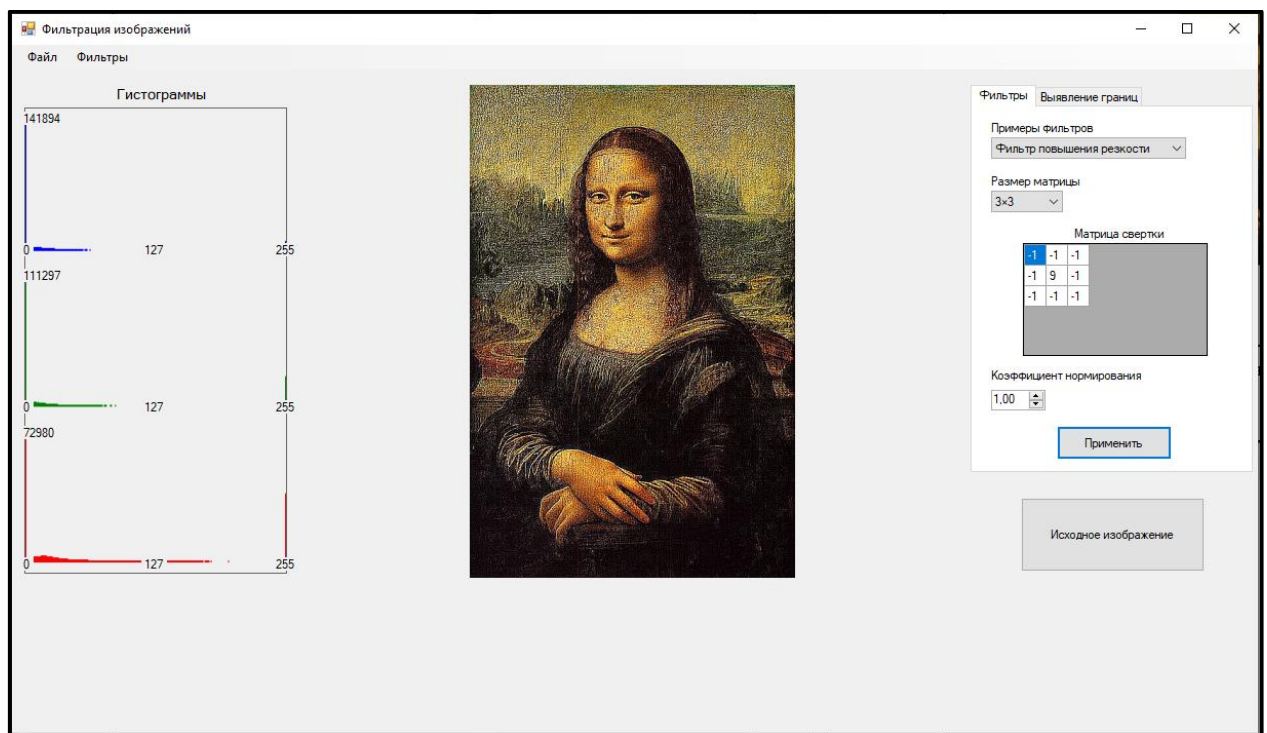


Рисунок 5 – Применяем фильтр резкости

Фильтры выявления границ:

До применения фильтра:

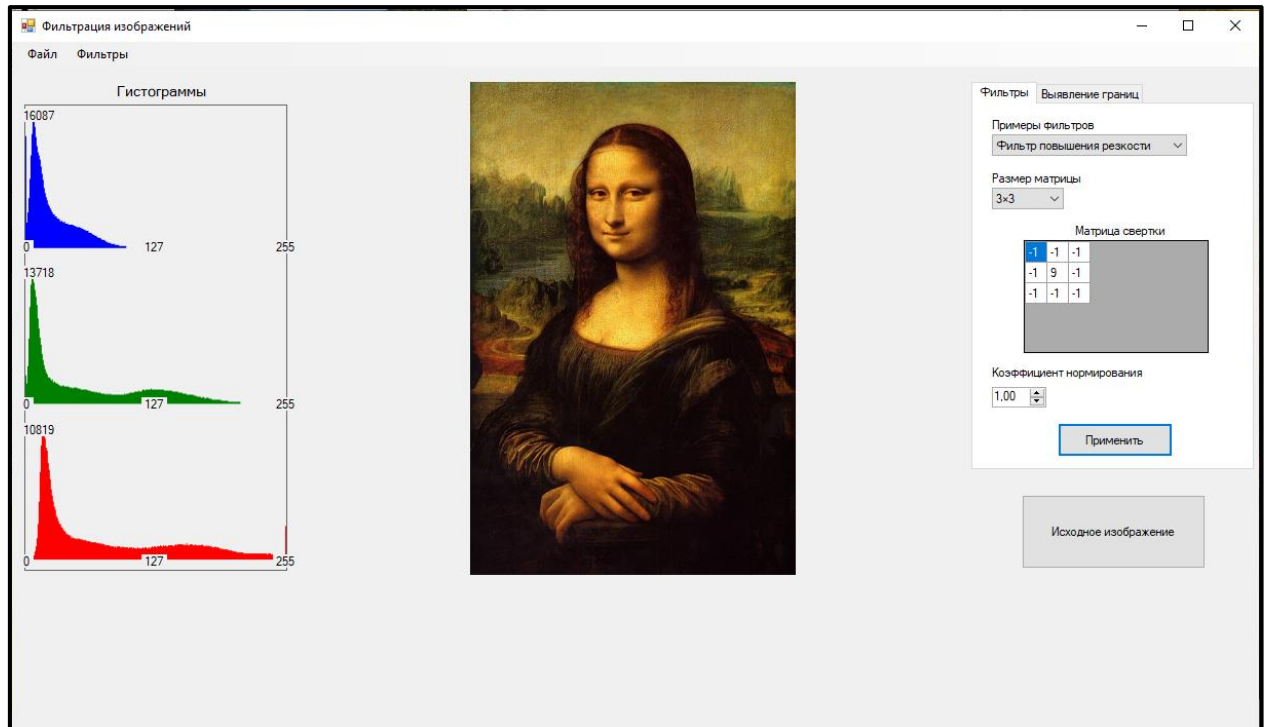


Рисунок 6 – Оригинальное изображение

Фильтр Робертса

После применения фильтра:



Рисунок 7 – Фильтр Робертса (предел= 80)

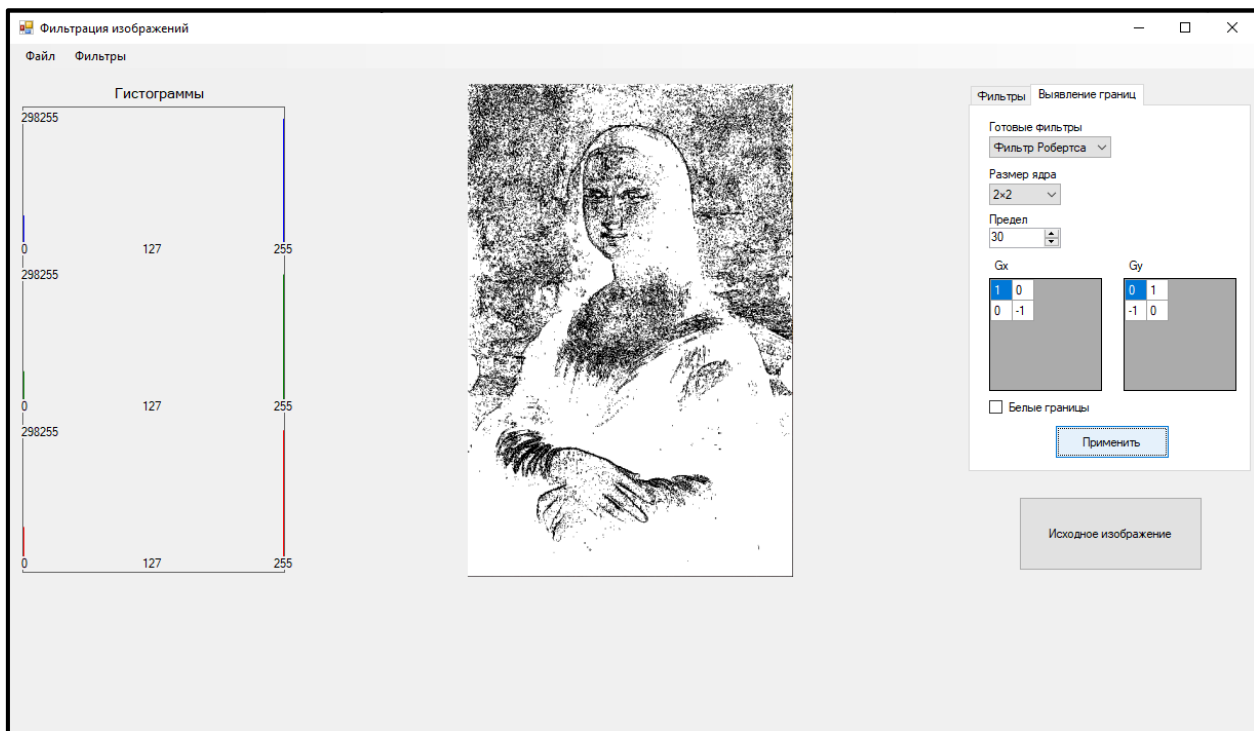


Рисунок 8 – Фильтр Роберта (предел= 30)

Фильтр Превитта

После применения фильтра:

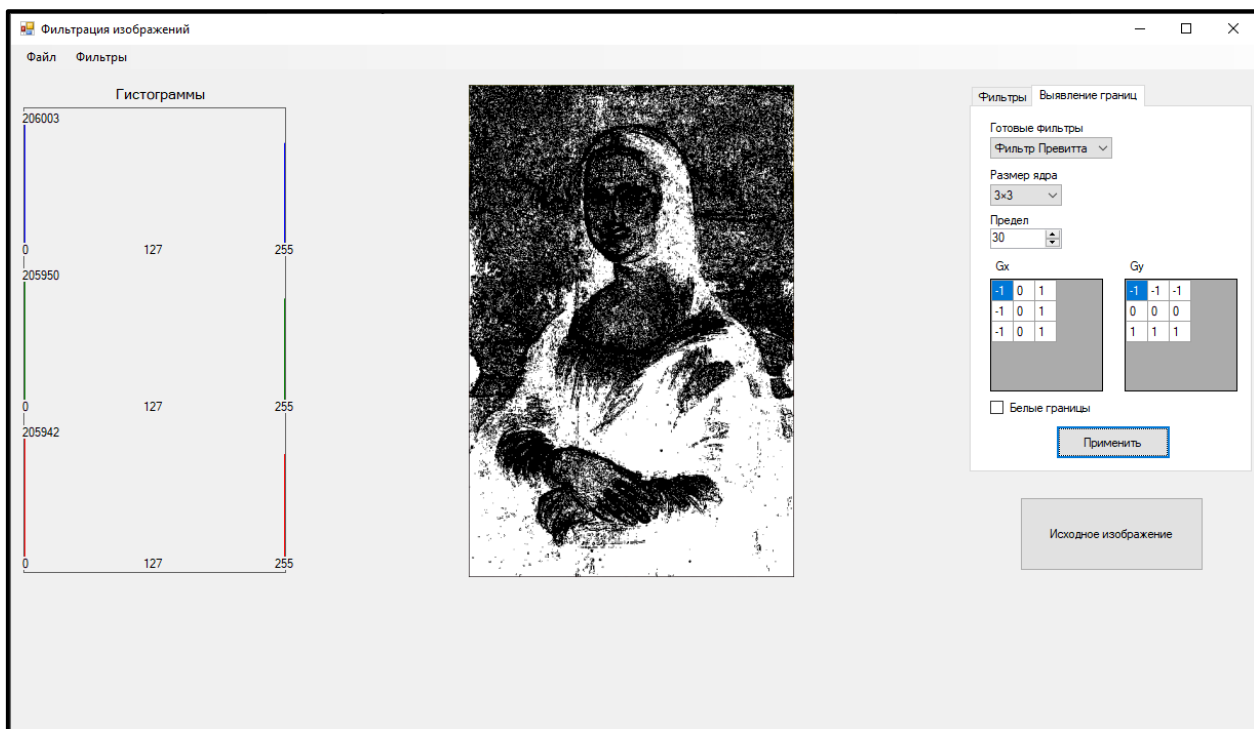


Рисунок 9 – Фильтр Превитта

Фильтр Собеля

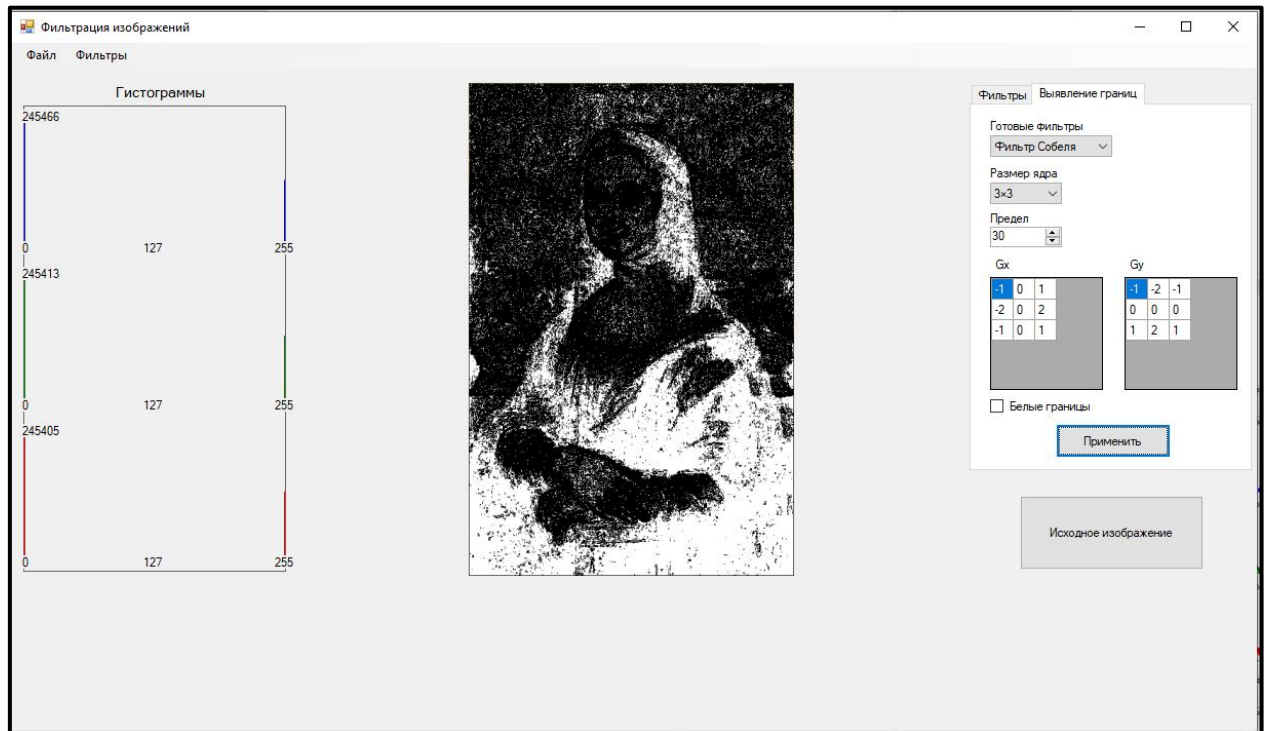


Рисунок 10 – Фильтр Собеля

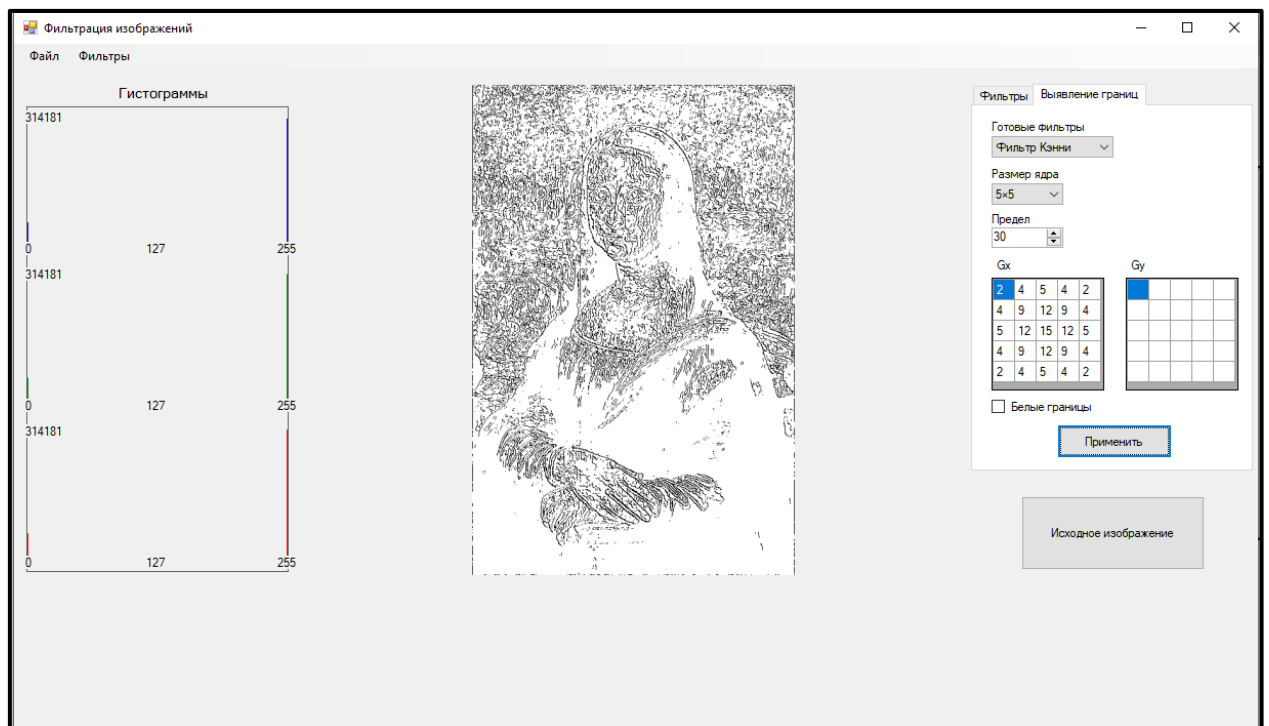


Рисунок 11 – Фильтр Кэнни

Фильтр повышения яркости

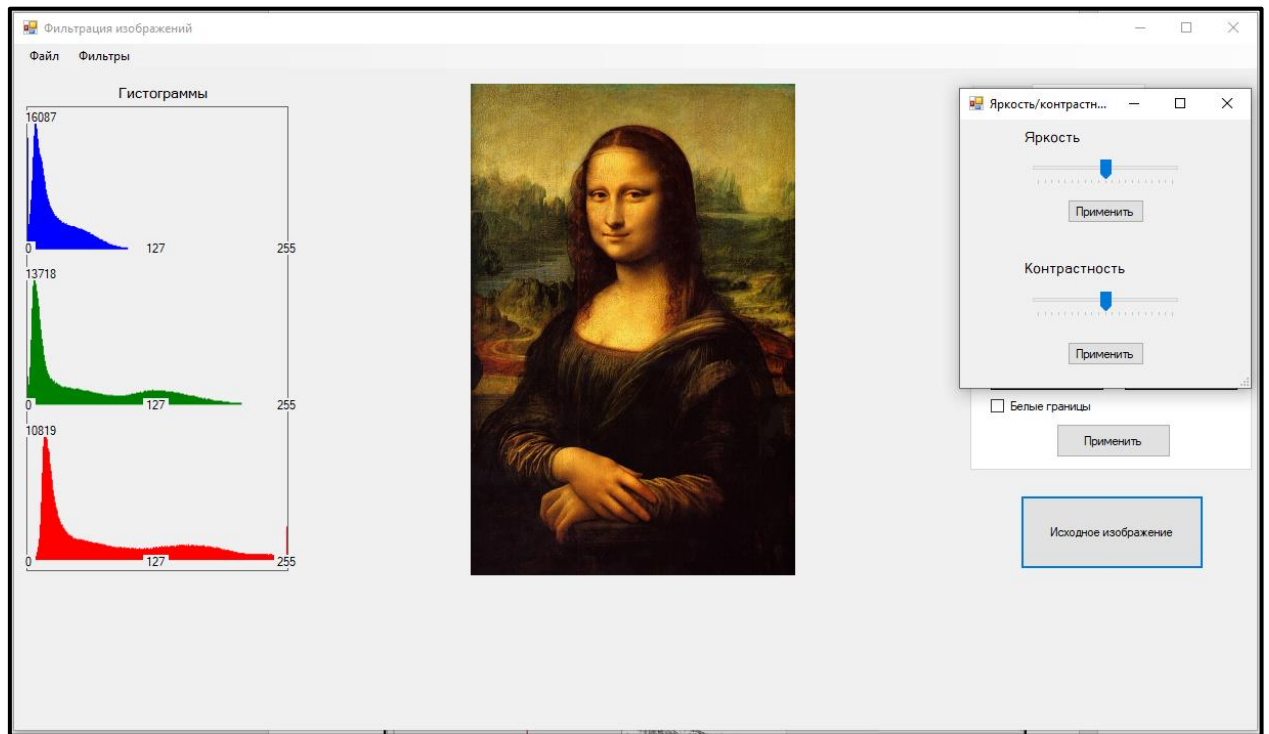


Рисунок 12 – Оригинальное изображение

После увеличения яркости до максимума:

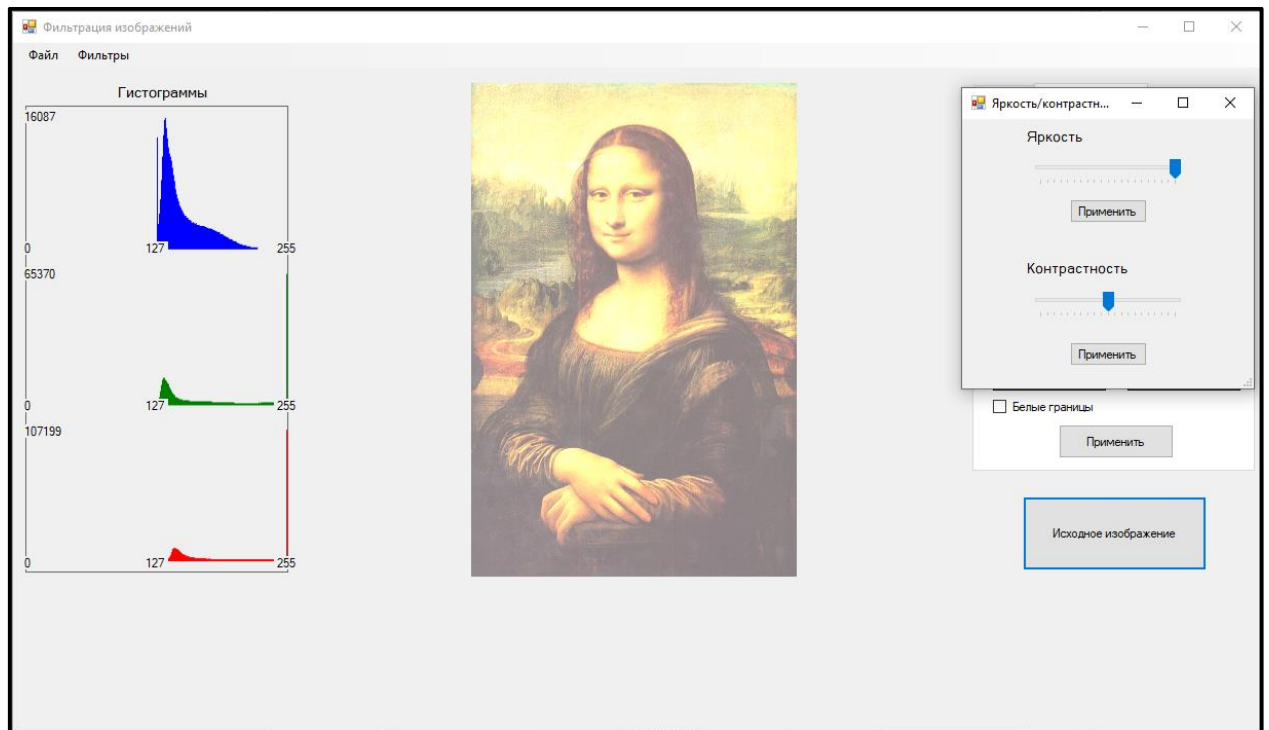


Рисунок 13 – Яркость на максимум

После увеличения контрастности до максимума:

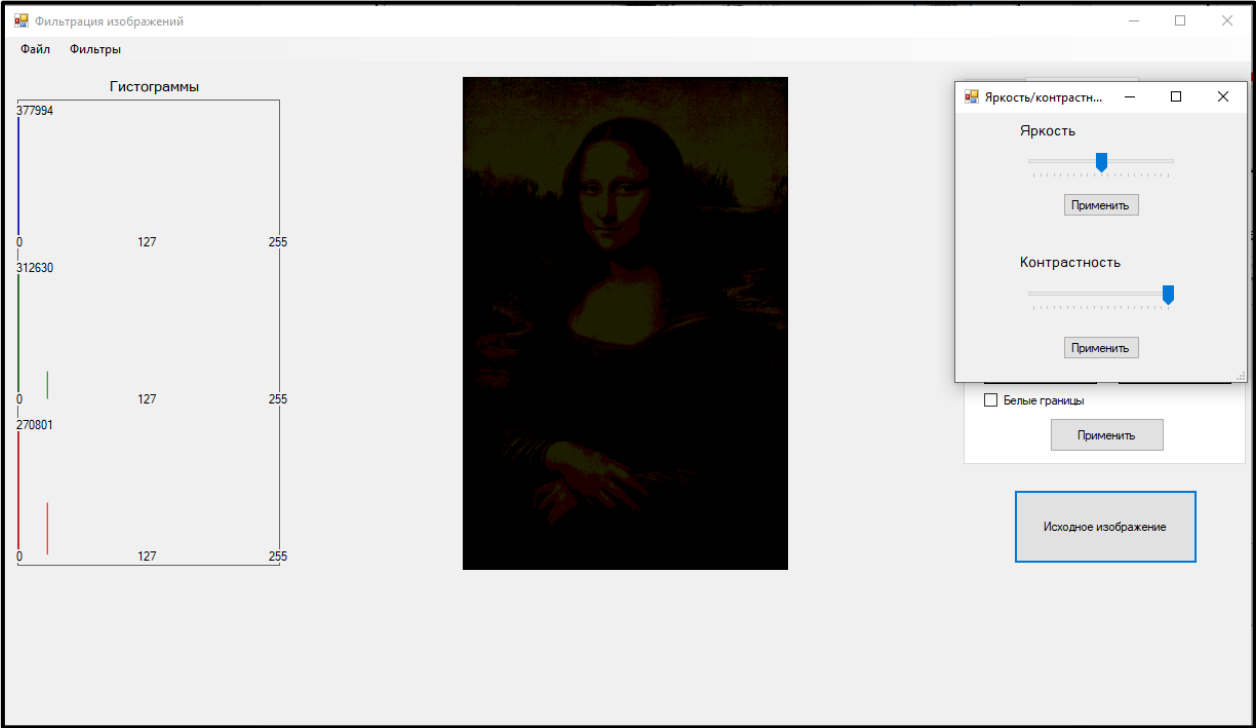


Рисунок 14 – Контрастность на максимум



Рисунок 15 – Контрастность на минимум

Вывод по фильтрам:

В ходе изучения программы были изучены различные фильтры обработки изображений, их особенности и влияние на визуальное представление данных. Основные результаты можно сформулировать следующим образом:

1. Фильтр размытия:

- Эффективно снижает уровень шума и мелких деталей на изображении.
- Полезен для предварительной обработки перед применением других фильтров, например, для сглаживания границ.

2. Фильтр резкости:

- Увеличивает четкость изображения, подчеркивая детали.
- Может быть полезен для улучшения видимости мелких элементов, но при чрезмерном применении приводит к появлению артефактов.

3. Фильтры выявления границ:

- **Фильтр Робертса:** Хорошо выделяет границы, но чувствителен к шуму. Изменение порога (например, с 80 до 30) позволяет регулировать количество деталей.
- **Фильтр Превитта:** Менее чувствителен к шуму по сравнению с Робертсом, но также эффективно выделяет границы.
- **Фильтр Собеля:** Обеспечивает более плавное и точное выделение границ, часто используется в комбинации с другими методами.

4. Фильтр повышения яркости и контрастности:

- Увеличение яркости делает изображение светлее, что полезно для темных сцен.
- Увеличение контрастности усиливает разницу между светлыми и темными областями, улучшая восприятие деталей. Однако чрезмерное увеличение может привести к потере информации в слишком светлых или темных участках.

Функция Canny()

Довести до рабочего состояния программу, приведенную ниже. Провести 2-3 эксперимента с разными значениями параметров (порогов) функции Canny()

Код:

```
import cv2
# Загрузка изображения в оттенках серого
img = cv2.imread('image.jpg',0)
# Применение алгоритма Canny для обнаружения границ
edges = cv2.Canny(img,100,200)
# Вывод изображения с границами
cv2.imshow('Edges',edges)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Доработанный код:

```
import cv2
import numpy as np

def apply_canny(image_path, threshold1, threshold2):
    # Загрузка изображения в оттенках серого
    img = cv2.imread(image_path, 0)

    # Проверка успешной загрузки изображения
    if img is None:
        print(f"Ошибка: Не удалось загрузить изображение {image_path}")
        return

    # Применение алгоритма Canny для обнаружения границ
    edges = cv2.Canny(img, threshold1, threshold2)

    # Создание окна с результатами
    result = np.hstack((img, edges))

    # Вывод изображения с границами
    cv2.imshow(f'Original vs Canny (Thresholds: {threshold1}, {threshold2})',
result)
    cv2.waitKey(0)
    cv2.destroyAllWindows()

# Основная часть программы
if __name__ == "__main__":
    image_path = 'image.jpg' # Убедитесь, что изображение лежит в той же папке

    # Эксперимент 1: Низкие пороги (более чувствительный детектор)
    apply_canny(image_path, 50, 100)

    # Эксперимент 2: Средние пороги (баланс между шумом и детализацией)
    apply_canny(image_path, 100, 200)

    # Эксперимент 3: Высокие пороги (меньше шума, но пропускает слабые границы)
    apply_canny(image_path, 150, 250)
```

1. Низкие пороги (более чувствительный детектор):

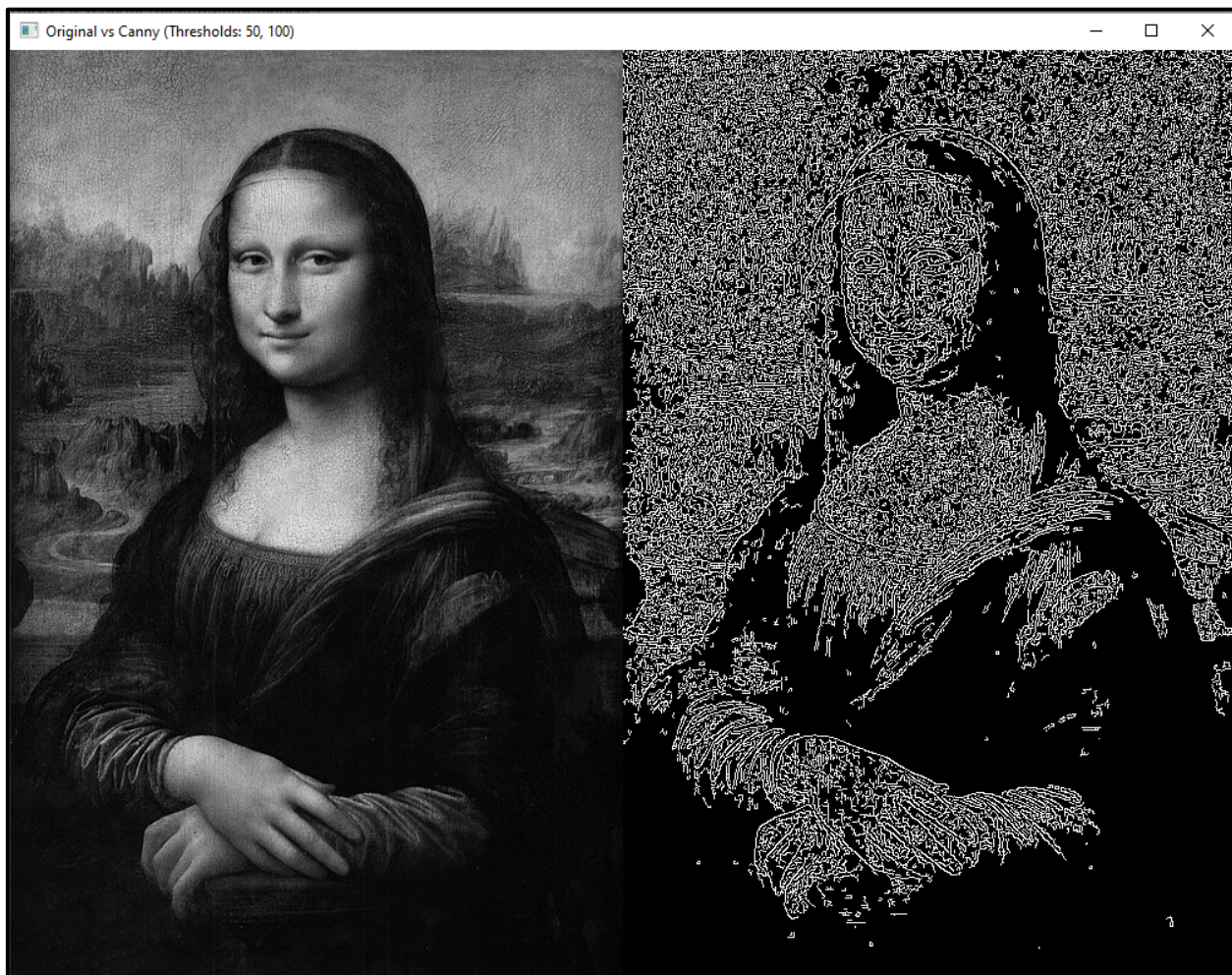


Рисунок 16 – Оригинальное изображение / Изображение с низкими порогами

2. Средние пороги (баланс между шумом и детализацией)

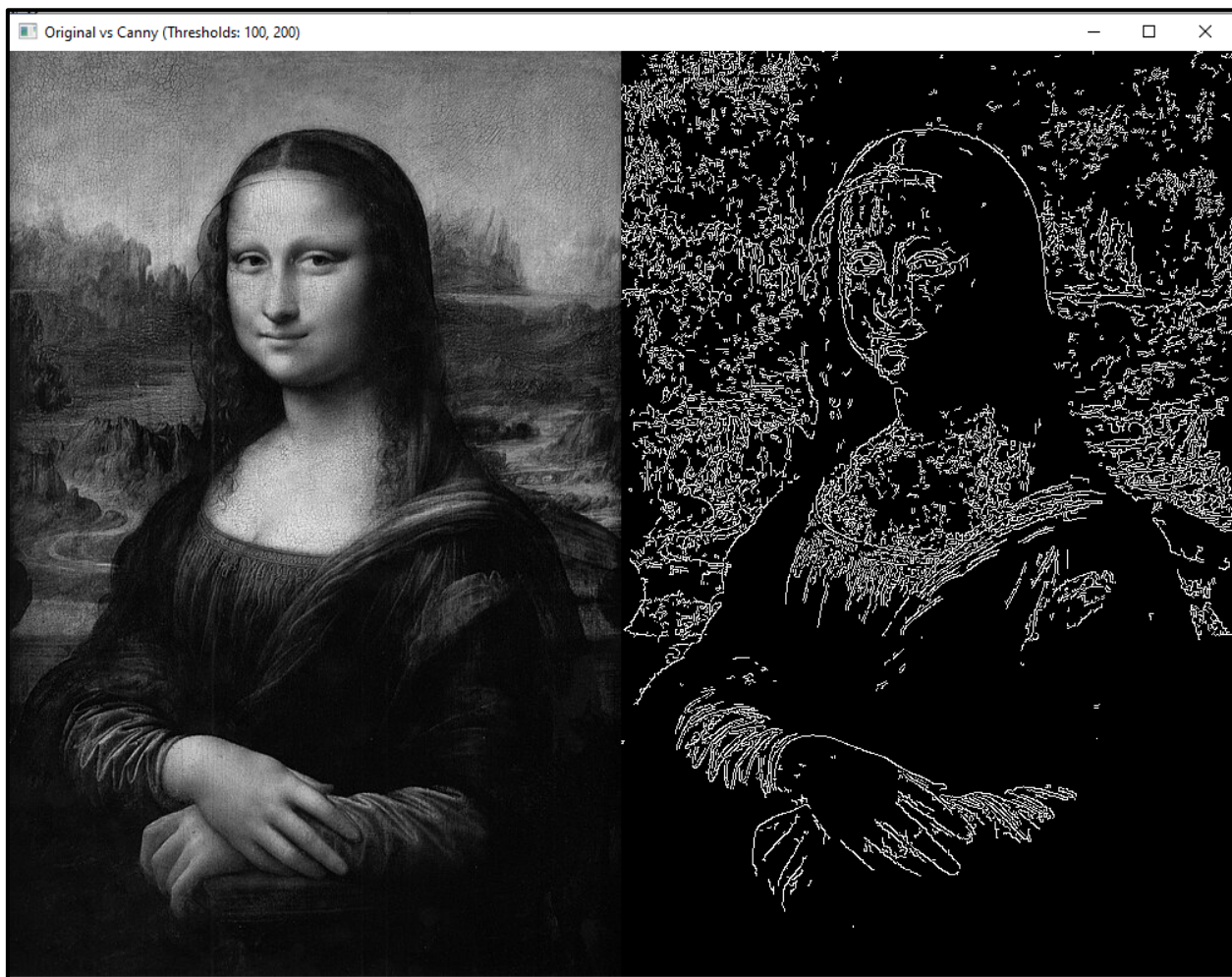


Рисунок 17 – Оригинальное изображение / Сбалансированное изображение

3. Высокие пороги (меньше шума, но пропускает слабые границы)

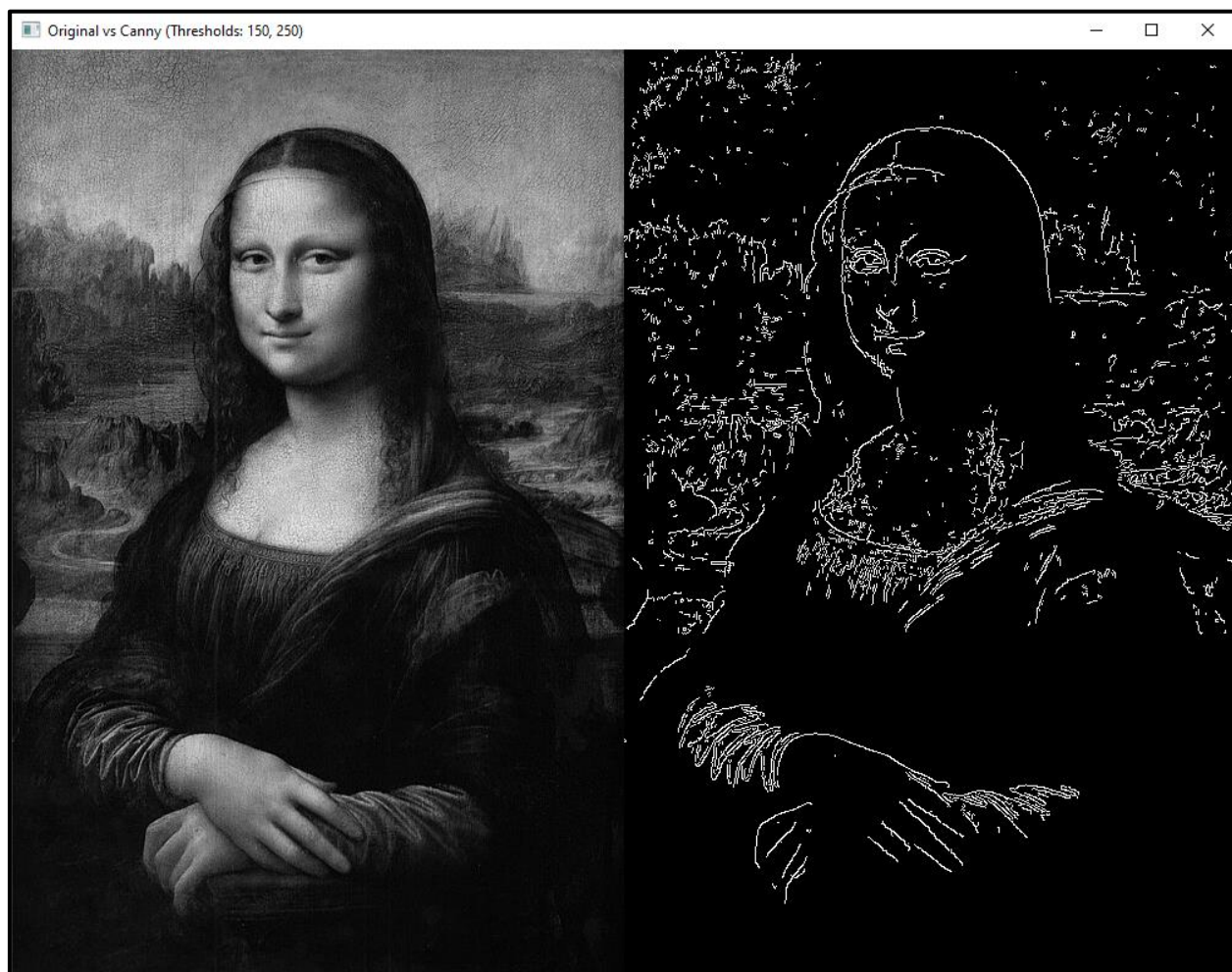


Рисунок 18 – Оригинальное изображение / Изображение с высокими порогами

Вывод по эксперименту с фильтром Кэнни (Canny):

В ходе выполнения лабораторной работы были проведены эксперименты с алгоритмом обнаружения границ Кэнни, использующим разные пороговые значения. Основные результаты и наблюдения:

1. Влияние пороговых значений на результат

- **Низкие пороги (50, 100)** – алгоритм выделяет множество границ, включая слабые и шумовые. Это полезно для детектирования мелких объектов, но приводит к избыточному количеству ложных срабатываний.
- **Средние пороги (100, 200)** – обеспечивают баланс между детализацией и уровнем шума. Большинство значимых границ сохраняются, а слабые и шумовые отфильтровываются.
- **Высокие пороги (150, 250)** – алгоритм оставляет только самые четкие границы, что снижает уровень шума, но может привести к потере важных деталей.

2. Оптимальные настройки

- Для большинства задач подходят **пороги 100–200**, так как они дают достаточно четкие границы без избыточного шума.
- Если изображение содержит много мелких деталей, можно **понизить пороги**, но затем дополнительно обработать результат (например, морфологическими операциями).
- Для изображений с высокой контрастностью и минимумом шума можно **повысить пороги**, чтобы получить более строгий результат.